

DDA5001 Machine Learning

Neural Networks (Part III): SGD and Adam

Yuning You

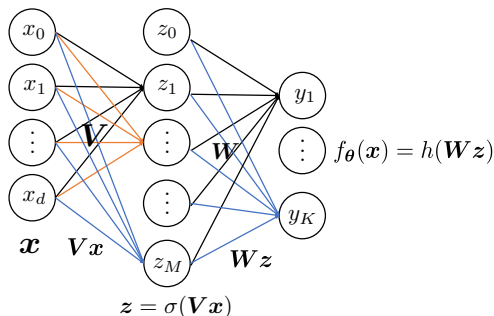
School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Recap: Forward Pass in Backpropagation

Forward pass: Feed x_i into the network, compute and **store**

- ▶ Vx_i
- ▶ $z_i = \sigma(Vx_i)$
- ▶ Wz_i
- ▶ $f_{\theta}(x_i) = h(Wz_i)$



Recap: Backward Pass in Backpropagation

Backward pass:

Compute the gradient of the second layer

$$\blacktriangleright \delta_i = 2(h(\mathbf{W}\mathbf{z}_i) - \mathbf{y}_i) \odot h'(\mathbf{W}\mathbf{z}_i)$$

$$\blacktriangleright \frac{\partial \ell_i}{\partial \mathbf{W}} = \delta_i \mathbf{z}_i^\top$$

“Backpropagate” to the first layer and compute

$$\blacktriangleright \mathbf{W}^\top \delta_i$$

$$\blacktriangleright \frac{\partial \ell_i}{\partial \mathbf{V}} = \left((\mathbf{W}^\top \delta_i) \odot \sigma'(\mathbf{V}\mathbf{x}_i) \right) \mathbf{x}_i^\top$$

Thus, **backpropagation** is an efficient way of computing the gradient of NN learning problem.

Given the computed gradient, we can apply gradient-based training algorithm for training NN. $\rightsquigarrow$ Which algorithm should we choose?

Agenda

Stochastic Gradient Descent (SGD)

Adam Family

Stochastic Gradient Descent

Consider $\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell_i(\boldsymbol{\theta})$, where n is very large.

The idea: Can we just use gradient information of **only one component function** ℓ_i rather than the full gradient for algorithmic update?

Stochastic Gradient Descent

Consider $\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell_i(\boldsymbol{\theta})$, where n is very large.

The idea: Can we just use gradient information of **only one component function** ℓ_i rather than the full gradient for algorithmic update?

Algorithmic design framework

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \left\{ q_k(\boldsymbol{\theta}) + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2 \right\}$$

- ▶ The q_k is the linear approximation of $\mathcal{L}$ using the gradient of **only one** component function

$$q_k(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}_k) + \langle \nabla \ell_{i_k}(\boldsymbol{\theta}_k), \boldsymbol{\theta} - \boldsymbol{\theta}_k \rangle,$$

ℓ_{i_k} is the i_k -th component function at k -th iteration, $\nabla \ell_{i_k}(\boldsymbol{\theta}_k)$ is called **stochastic gradient**.

Stochastic Gradient Descent

Consider $\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell_i(\boldsymbol{\theta})$, where n is very large.

The idea: Can we just use gradient information of **only one component function** ℓ_i rather than the full gradient for algorithmic update?

Algorithmic design framework

$$\boldsymbol{\theta}_{k+1} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \left\{ q_k(\boldsymbol{\theta}) + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2 \right\}$$

- The q_k is the linear approximation of $\mathcal{L}$ using the gradient of **only one** component function

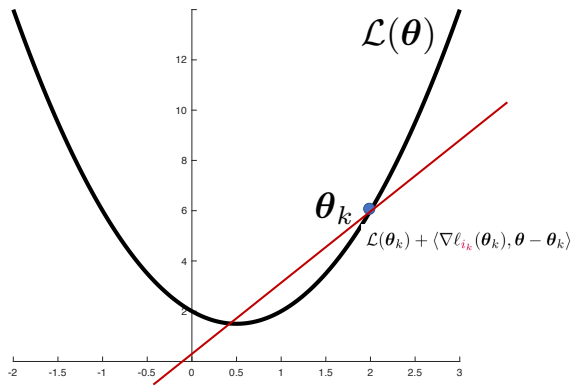
$$q_k(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}_k) + \langle \nabla \ell_{i_k}(\boldsymbol{\theta}_k), \boldsymbol{\theta} - \boldsymbol{\theta}_k \rangle,$$

ℓ_{i_k} is the i_k -th component function at k -th iteration, $\nabla \ell_{i_k}(\boldsymbol{\theta}_k)$ is called **stochastic gradient**.

Stochastic gradient descent: SGD

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \ell_{i_k}(\boldsymbol{\theta}_k).$$

SGD: Geometric Illustration



SGD vs. GD

GD:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \underbrace{\nabla \mathcal{L}(\boldsymbol{\theta}_k)}_{\frac{1}{n} \sum_{i=1}^n \nabla \ell_i(\boldsymbol{\theta}_k)}$$

- ▶ Computationally expensive iteration when n is very large.
- ▶ Opportunities for parallelism.

SGD:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \ell_{i_k}(\boldsymbol{\theta}_k)$$

- ▶ Very cheap iteration.
- ▶ Sequential use of data, nonparallel.

SGD vs. GD

GD:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \underbrace{\nabla \mathcal{L}(\boldsymbol{\theta}_k)}_{\frac{1}{n} \sum_{i=1}^n \nabla \ell_i(\boldsymbol{\theta}_k)}$$

- ▶ Computationally expensive iteration when n is very large.
- ▶ Opportunities for parallelism.

SGD:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \ell_{i_k}(\boldsymbol{\theta}_k)$$

- ▶ Very cheap iteration.
- ▶ Sequential use of data, nonparallel.

Efficiency of SGD over GD:

- ▶ Assume all the n training data $(\mathbf{x}_i, y_i)$ are the copies of one data (somewhat extreme).
- ▶ GD is n times more expensive than SGD.

The Choice of i_k

SGD:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \ell_{i_k}(\boldsymbol{\theta}_k)$$

Parameters in SGD needed to be preset:

- ▶ The usual ones: learning rate μ_k , initial point $\boldsymbol{\theta}_0$, etc.
- ▶ The additional one: The component function **index** i_k at each iteration.

The Choice of i_k

SGD:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \ell_{i_k}(\boldsymbol{\theta}_k)$$

Parameters in SGD needed to be preset:

- ▶ The usual ones: learning rate μ_k , initial point $\boldsymbol{\theta}_0$, etc.
- ▶ The additional one: The component function **index** i_k at each iteration.

Random choice: At k -th iteration, choose i_k uniformly at random from $\{1, \dots, n\}$.

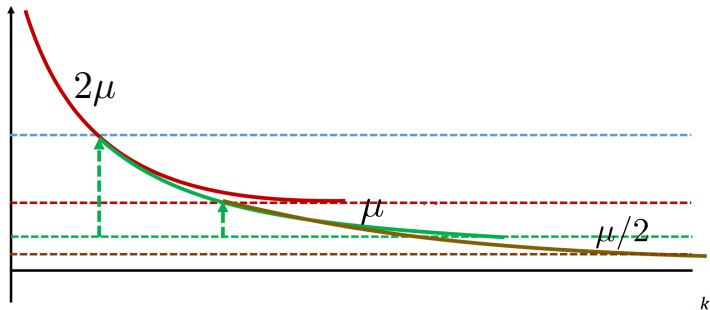
Theoretical justification: **Unbiased** approximation of the full gradient.

$$\mathbb{E}_{i_k} [\nabla \ell_{i_k}(\boldsymbol{\theta}_k)] = \frac{1}{n} \sum_{i=1}^n \nabla \ell_i(\boldsymbol{\theta}_k) = \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

- ▶ In expectation, SGD is GD.

Learning Rate μ_k in SGD

- ▶ One choice is $\mu_k = \frac{c}{\sqrt{k}}$ (similar to subgradient method, **SGD is a non-descent method**. Hence, we need to use diminishing learning rate in SGD).
- ▶ Apart from the diminishing learning rate $\mu_k = \frac{c}{\sqrt{k}}$, **step-decay** is also utilized in practice.



- ▶ Step-decay: Whenever progress slows down, we **reduce the learning rate by half** and continue the update.

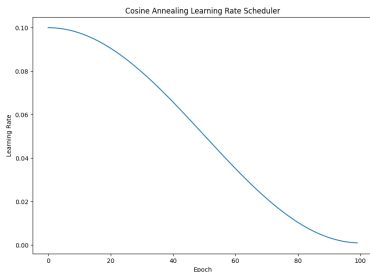
Cosine (Annealing) Scheduler Learning Rate

Recently, there is another famous and widely used learning rate schedule, called **cosine scheduler**. It requires to specify:

- ▶ The maximum initial learning rate μ_0 .
- ▶ The minimal final learning rate $\mu_{\min}$.
- ▶ The preset total number of iteration T .

Then, cosine scheduler is to **automatically decay** μ_0 to $\mu_{\min}$ in T iterations:

$$\mu_k = \mu_{\min} + \frac{1}{2}(\mu_0 - \mu_{\min}) \left(1 + \cos \left(\frac{k}{T} \pi \right) \right).$$



Application: Text Classification with SVM

► Dataset

- RCV1 document corpus.
- 781265 training examples ($= n$), 23149 testing examples.
- 47152 features ($= d$).

► Task

- Recognizing documents of category CCAT.

► Machine Learning model: Train soft-margin SVM classifier

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d, b \in \mathbb{R}} \mathcal{L}(\boldsymbol{\theta}) := \frac{1}{n} \sum_{i=1}^n \left\{ \max(0, 1 - y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b)) + \lambda \|\boldsymbol{\theta}\|_2^2 \right\}$$

	Training Time	Primal cost	Test Error
SVMLight	23,642 secs	0.2275	6.02%
SVMPerf	66 secs	0.2278	6.03%
SGD	1.4 secs	0.2275	6.02%

▪

Random Reshuffling

Random Reshuffling: The More Practical SGD

- ▶ The uniformly random choice of i_k in SGD **is not practical**. It is mainly for theoretical analysis of SGD.
- ▶ In SGD, we need to generate a random number i_k from $\{1, \dots, n\}$ **at each iteration**. This can be time costly.
- ▶ In addition, uniformly random sampling **may not** utilize the training data fully in **one epoch**. Here, one epoch implies n iterations of SGD (one epoch means utilizing n data points, hence n iterations).

Random Reshuffling: The More Practical SGD

- ▶ The uniformly random choice of i_k in SGD is not practical. It is mainly for theoretical analysis of SGD.
- ▶ In SGD, we need to generate a random number i_k from $\{1, \dots, n\}$ at each iteration. This can be time costly.
- ▶ In addition, uniformly random sampling may not utilize the training data fully in one epoch. Here, one epoch implies n iterations of SGD (one epoch means utilizing n data points, hence n iterations).

Random Reshuffling (RR)

In k -th epoch, we first uniformly at random permute the order $\{1, \dots, n\}$ to get σ_k . Then, we update by sequentially visiting the component function indexed by σ_k . The resulting algorithm is called Random Reshuffling (RR).

Random Reshuffling: The More Practical SGD

- ▶ The uniformly random choice of i_k in SGD is not practical. It is mainly for theoretical analysis of SGD.
- ▶ In SGD, we need to generate a random number i_k from $\{1, \dots, n\}$ at each iteration. This can be time costly.
- ▶ In addition, uniformly random sampling may not utilize the training data fully in one epoch. Here, one epoch implies n iterations of SGD (one epoch means utilizing n data points, hence n iterations).

Random Reshuffling (RR)

In k -th epoch, we first uniformly at random permute the order $\{1, \dots, n\}$ to get σ_k . Then, we update by sequentially visiting the component function indexed by σ_k . The resulting algorithm is called Random Reshuffling (RR).

Random Reshuffling: The More Practical SGD

- ▶ The uniformly random choice of i_k in SGD is not practical. It is mainly for theoretical analysis of SGD.
- ▶ In SGD, we need to generate a random number i_k from $\{1, \dots, n\}$ at each iteration. This can be time costly.
- ▶ In addition, uniformly random sampling may not utilize the training data fully in one epoch. Here, one epoch implies n iterations of SGD (one epoch means utilizing n data points, hence n iterations).

Random Reshuffling (RR)

In k -th epoch, we first uniformly at random permute the order $\{1, \dots, n\}$ to get σ_k . Then, we update by sequentially visiting the component function indexed by σ_k . The resulting algorithm is called Random Reshuffling (RR).

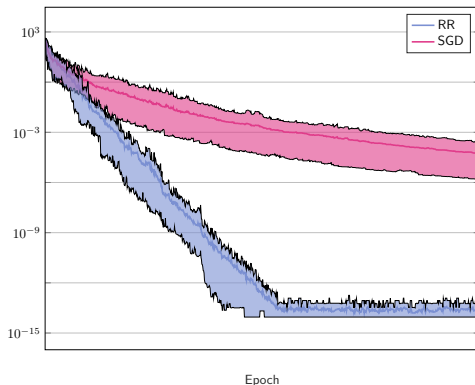
RR vs SGD: Suppose $n = 5$, one epoch of SGD and RR may be

RR	f_2	f_5	f_4	f_1	f_3
SGD	f_3	f_2	f_2	f_1	f_3

- ▶ RR uses all the samples in one epoch.

Random Reshuffling: Experiment

Implement two algorithms on simple LS.



In summary:

- ▶ RR converges much **faster** than SGD empirically.
- ▶ RR is easy to implement due to **simple permutation in one epoch**.
- ▶ What is commonly implemented in practice is indeed RR; people just call it SGD. Check PyTorch's SGD implementation, it is RR.

SGD with Momentum

SGD with Momentum

We can also include momentum acceleration technique to speed up the convergence speed of SGD.

SGD with momentum:

$$\begin{aligned}\boldsymbol{\theta}_{k+1} &= \boldsymbol{\theta}_k - \mathbf{m}_k \\ \mathbf{m}_k &= \mu_k \nabla \ell_{i_k}(\boldsymbol{\theta}_k) + \beta_k \mathbf{m}_{k-1}\end{aligned}$$

SGD with Nesterov's momentum:

$$\begin{aligned}\boldsymbol{\theta}_{k+1} &= \mathbf{w}_k - \mu_k \nabla \ell_{i_k}(\mathbf{w}_k) \\ \mathbf{w}_k &= \boldsymbol{\theta}_k + \frac{k-1}{k+2}(\boldsymbol{\theta}_k - \boldsymbol{\theta}_{k-1})\end{aligned}$$

SGD with momentum can substantially improve the training speed compared to SGD only.

Agenda

Stochastic Gradient Descent (SGD)

Adam Family

Adaptive Learning Rate Method: AdaGrad

Motivations:

- ▶ All the optimization algorithms we have studied so far assign the same learning rate to all coordinates/elements in θ .
- ▶ We might need different learning rates for different coordinates in θ .

Adaptive Learning Rate Method: AdaGrad

Motivations:

- ▶ All the optimization algorithms we have studied so far assign the same learning rate to all coordinates/elements in θ .
- ▶ We might need different learning rates for different coordinates in θ .

AdaGrad is an important method that utilizes adaptive learning rate.

- ▶ The idea: Normalize each gradient coordinate by the past gradients.
- ▶ It has the following update:

$$\theta_{k+1} = \theta_k - \mu_k \frac{\mathbf{g}_k}{\sqrt{\sum_{t=1}^k \mathbf{g}_t^2}},$$

where $\mathbf{g}_k = \nabla \ell_{i_k}(\theta_k)$ is the stochastic gradient. The update is performed element-wise (separately for each coordinate):

$$\theta_{k+1}[j] = \theta_k[j] - \mu_k \frac{\mathbf{g}_k[j]}{\sqrt{\sum_{t=1}^k (\mathbf{g}_t[j])^2}}, j = 1, \dots, d.$$

- ▶ AdaGrad penalizes gradient coordinates with large values, while increasing small ones for exploration.

AdaGrad reduces the burden for tuning learning rate, and it is often more reliable than SGD.

Adam: Momentum Meets Adaptive Learning Rate

Observations:

- ▶ SGD with momentum can improve convergence speed.
- ▶ Adaptive learning rate like AdaGrad can reduce the need for learning rate tuning and stabilize convergence.

Adam: Momentum Meets Adaptive Learning Rate

Observations:

- ▶ SGD with momentum can improve convergence speed.
- ▶ Adaptive learning rate like AdaGrad can reduce the need for learning rate tuning and stabilize convergence.

How about combining these two techniques?

Adam: Momentum Meets Adaptive Learning Rate

Observations:

- ▶ SGD with momentum can improve convergence speed.
- ▶ Adaptive learning rate like AdaGrad can reduce the need for learning rate tuning and stabilize convergence.

How about combining these two techniques? $\rightsquigarrow$ Adam method.

Adam: Momentum Meets Adaptive Learning Rate

Observations:

- ▶ SGD with momentum can improve convergence speed.
- ▶ Adaptive learning rate like AdaGrad can reduce the need for learning rate tuning and stabilize convergence.

How about combining these two techniques? $\rightsquigarrow$ Adam method.

- ▶ Adam uses **moving averaging** for updating the momentum and adaptive learning rate.

Momentum update:

$$\mathbf{m}_k = \beta_1 \mathbf{m}_{k-1} + (1 - \beta_1) \mathbf{g}_k, \quad \text{with } \mathbf{m}_0 = 0.$$

Adaptive learning rate update:

$$\mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) \mathbf{g}_k^2, \quad \text{with } \mathbf{v}_0 = 0.$$

$\mathbf{g}_k = \nabla \ell_{i_k}(\boldsymbol{\theta}_k)$ is the stochastic gradient.

Rule of thumb: $\beta_1 = 0.9$, $\beta_2 = 0.999$.

Understanding Moving Averaging

If we expand the update of momentum and adaptive learning rate in Adam, we will have

$$\mathbf{m}_k = \beta_1^k \mathbf{m}_0 + \beta_1^{k-1}(1 - \beta_1) \mathbf{g}_1 + \cdots + \beta_1(1 - \beta_1) \mathbf{g}_{k-1} + (1 - \beta_1) \mathbf{g}_k,$$

and

$$\mathbf{v}_k = \beta_2^k \mathbf{v}_0 + \beta_2^{k-1}(1 - \beta_2) \mathbf{g}_1^2 + \cdots + \beta_2(1 - \beta_2) \mathbf{g}_{k-1}^2 + (1 - \beta_2) \mathbf{g}_k^2,$$

Interpretations: The moving averaging **exponentially** decays the importance of the historical records, while **emphasizing more on the most recent ones**.

Adam: Algorithmic Procedure

Adam

1. Compute a stochastic gradient $\mathbf{g}_k = \nabla \ell_{i_k}(\boldsymbol{\theta}_k)$;
2. Update momentum: $\mathbf{m}_k = \beta_1 \mathbf{m}_{k-1} + (1 - \beta_1) \mathbf{g}_k$;
3. Update adaptive learning rate $\mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) \mathbf{g}_k^2$;
4. Bias-corrected momentum: $\mathbf{m}_k = \mathbf{m}_k / (1 - \beta_1^k)$;
5. Bias-corrected adaptive learning rate: $\mathbf{v}_k = \mathbf{v}_k / (1 - \beta_2^k)$;
6. Update:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \frac{\mathbf{m}_k}{\sqrt{\mathbf{v}_k + 10^{-8}}}.$$

Repeat until convergence.

- ▶ The bias-correction term is used to make $\mathbf{m}_k$ and $\mathbf{v}_k$ unbiased estimators for $\mathbb{E}[g_k]$ and $\mathbb{E}[g_k^2]$. **Not sure** if this step is crucial for practical performance.
- ▶ Adam is widely used in practice for training NN.
- ▶ Similar to AdaGrad, the updates in Adam are also performed **element-wise**, i.e., separately for each coordinate.

Weight Decay Issue in Adam

Weight decay regularization is widely utilized in NN training, since otherwise it easily suffers from overfitting due to the strong representation power of NN.

- ▶ Therefore, Adam is often used together with weight decay.
- ▶ However, traditionally, the way in which people apply weight decay is through **changing the stochastic gradient**:

$$\mathbf{g}_k = \nabla \ell_{i_k}(\boldsymbol{\theta}_k) + \lambda \boldsymbol{\theta}_k.$$

Recall that the gradient of ℓ_2 regularization term $\frac{\lambda}{2} \|\boldsymbol{\theta}\|^2$ is $\lambda \boldsymbol{\theta}$.

Issues:

- ▶ $\mathbf{g}_k$ is not directly used for updating $\boldsymbol{\theta}_k$, it is used to update momentum and is normalized by the adaptive learning rate.
- ▶ If we add $\lambda \boldsymbol{\theta}_k$ to $\mathbf{g}_k$, then **the strength of regularization (and weight decay) also gets normalized**, which is undesirable.
- ▶ This makes it unclear whether we are actually decaying the weight.
- ▶ It is **no longer equivalent to the ℓ_2 -regularization** we studied in the regularization lecture.

AdamW: Decoupled Weight Decay

Idea: **Decouple** the weight decay term and use the true ℓ_2 -regularization.
 $\rightsquigarrow$ the **AdamW** method.

- ▶ All the steps of AdamW keeps the same as that of the Adam, except for the update:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \frac{\boldsymbol{m}_k}{\sqrt{\boldsymbol{v}_k + 10^{-8}}} - \mu_k \lambda \boldsymbol{\theta}_{k-1}.$$

- ▶ This additional $\lambda \boldsymbol{\theta}_{k-1}$ corresponds to the gradient of the weight decay term **added to training loss function directly**, namely, for NN training

$$\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2,$$

AdamW applies Adam for $\min \mathcal{L}$ while GD for $\min$ the weight decay regularization.

- ▶ **AdamW is truly applying ℓ_2 -regularization**. Thus, for huge NN training problems, AdamW often has **better generalization** than Adam, as better regularization improves generalization.

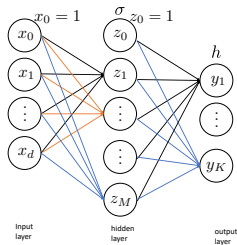
Experiments: Setup

We now conduct experiments on SGD, SGD momentum, SGD nesterov momentum, Adam, AdamW for training NN.

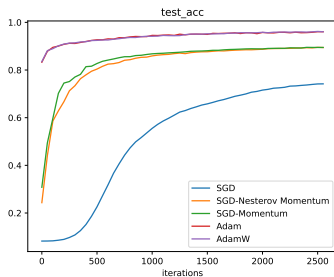
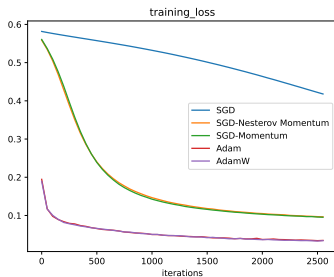
- **Task:** Classification on the MNIST dataset:



- **Model:** One hidden layer (two layer) NN, with $M = 64$, number of classes $K = 10$, σ is ReLU, h is softmax.



Experiments: Results¹



- ▶ SGD is slow.
- ▶ Adam family has the best performance, illustrating why they are quite ubiquitous in practice.

¹Code is available at <https://www.kaggle.com/code/jonery/lecture-demo>. You need to register Kaggle to use it.

Further Study and Reading

- ▶ Stochastic optimization methods are at the core of modern large-scale machine learning, serving as one of the foundations of deep learning.
- ▶ A very active research area in machine learning and optimization.

Further readings:

- ▶ Bottou, L., Curtis, F. E., Nocedal, J. (2018). Optimization methods for large-scale machine learning. *Siam Review*, 60(2), 223-311.
- ▶ Kingma, D. P., Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*
- ▶ Loshchilov, I., Hutter, F. (2017). Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*.

↪ Next lecture: Introductory deep learning.